

kan je voor alle unit testen die u geschreven heeft (waarvoor dank!) een README bestand neerschrijven?

Waarom en welke elementen zijn gekozen of gebruikt?

alsook enkele voorbeeld commando's met de beschikbare argumenten?

Natuurlijk! Hier is een voorbeeld van een **README** voor jouw unit tests, inclusief uitleg over keuzes, gebruikte elementen en voorbeeldcommando's.

README: Unit Tests voor MAC Spoofing Toolkit

Overzicht

Deze map bevat unit tests voor de Python MAC spoofing toolkit. De tests zijn ontworpen om de betrouwbaarheid, correctheid en platformonafhankelijke werking van de belangrijkste functies van de toolkit te waarborgen. Door deze tests regelmatig uit te voeren, kun je snel regressies of fouten in de functionaliteit opsporen.

Waarom unit tests?

- **Betrouwbaarheid:** Automatische tests zorgen ervoor dat wijzigingen aan de code niet per ongeluk bestaande functionaliteit breken.
- **Onderhoudbaarheid:** Tests maken het eenvoudiger om de code te refactoren of uit te breiden.
- **Documentatie:** Tests laten zien hoe functies bedoeld zijn te werken, wat helpt bij kennisoverdracht en samenwerking.
- **Snelle feedback:** Je weet direct of je code werkt zoals bedoeld, op verschillende platforms (Windows & Linux).

Gebruikte elementen en keuzes

1. unittest

- De standaard Python testmodule, gekozen vanwege brede ondersteuning en eenvoudige integratie.
- Structuur: elke testfunctie begint met `test_` en elke testklasse erft van `unittest.TestCase`.

2. unittest.mock (patch, MagicMock)

- Gebruikt om externe afhankelijkheden zoals systeemcommando's (`subprocess.run`) te vervangen door gesimuleerde (mock) objecten.
- Hierdoor worden er geen echte systeemwijzigingen uitgevoerd tijdens het testen.

3. platform-specifieke skips

- Sommige tests zijn alleen relevant op Windows of Linux. Met `@unittest.skipUnless` worden deze alleen uitgevoerd op het juiste platform.

4. sys.path-aanpassing

- In elk testbestand wordt het pad naar de hoofdmap toegevoegd, zodat de hoofdmodule (`mac_spoof.py`) altijd gevonden kan worden, onafhankelijk van de werkdirectory.

5. Testdekking

- Er zijn unittests voor:
 - Platformdetectie
 - MAC-adresgeneratie
 - Ophalen van huidige MAC-adressen (Linux/Windows)
 - Wijzigen van MAC-adressen (Linux)
 - Verificatie van MAC-wijziging
 - Interface-detectie
 - Subprocess-mocking

Voorbeeld: Structuur

```
saca_project/
├── mac_spoof.py
└── UNIT_TESTS_MAC_spoof/
    ├── __init__.py
    ├── test_mac_spoof.py
    ├── test_verify_mac_change_mac_spoof.py
    ├── test_get_available_interface_mac_spoof.py
    ├── test_subprocess_popen_mac_spoof.py
    └── test_conditional_skipping_mac_spoof.py
```

Voorbeeldcommando's

Alle tests automatisch uitvoeren

Voer deze commando's uit vanuit de hoofdmap (saca_project):

```
python -m unittest discover -s UNIT_TESTS_MAC_spoof
```

Eén specifiek testbestand uitvoeren

```
python -m unittest UNIT_TESTS_MAC_spoof.test_mac_spoof
```

Eén specifieke testklasse of testmethode uitvoeren

```
python -m unittest UNIT_TESTS_MAC_spoof.test_mac_spoof.TestMacSpoofeer
python -m unittest UNIT_TESTS_MAC_spoof.test_mac_spoof.TestMacSpoofeer.test_generate_mac
```

Output bij succesvolle tests

```
.....
-----
Ran 8 tests in 0.003s

OK (skipped=1)
```

Argumenten

- `-s <directory>` : Geef de directory op waar unittest moet zoeken naar testbestanden.
- `-p <pattern>` : Geef een patroon voor testbestandsnamen (standaard: `test*.py`).
- `<module>.<klasse>.<methode>` : Voer een specifieke test uit.

Best practice

- Plaats alle unittests in een aparte map, bijvoorbeeld `UNIT_TESTS_MAC_spoof`.
- Zorg dat elke testbestand begint met `test_` voor automatische herkenning.
- Voeg een leeg `__init__.py` toe aan de testmap.
- Gebruik mocks voor alle systeemcommando's en externe afhankelijkheden.

Afhankelijkheden

- Python 3.6 of hoger
- Geen extra packages nodig; alles is standaard in Python aanwezig

Opmerkingen

- De tests zijn bedoeld voor ontwikkel- en testdoeleinden. Ze voeren geen echte systeemwijzigingen uit.
- Platform-specifieke tests worden automatisch overgeslagen als ze niet van toepassing zijn.

Door deze unittests regelmatig uit te voeren, blijft je MAC spoofing toolkit betrouwbaar en onderhoudbaar!